Revealing Compiler Heuristics through Automated Discovery and Optimization

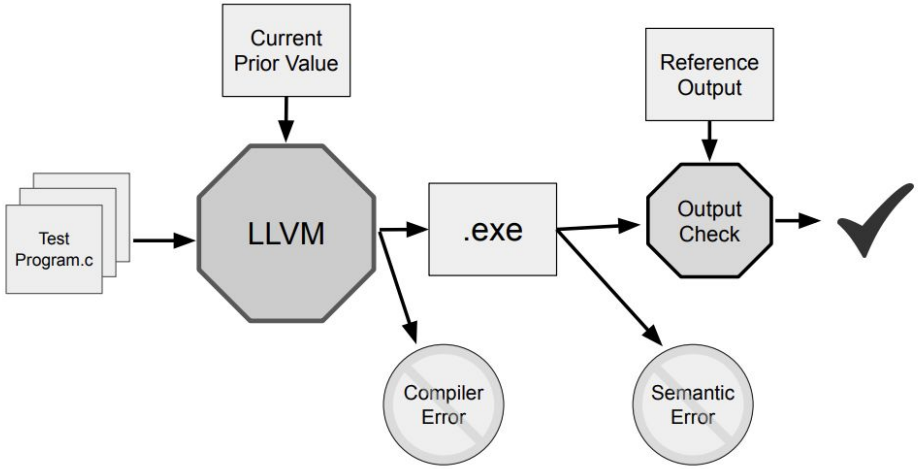
Shresth Tiwary Sharvani Sinha Fu-shiang Yang Terry Shi Jenny Xu

Revealing Compiler Heuristics through Automated Discovery and Optimization — CGO 2024

Motivation: Impractical to manually comb through millions of lines of code to find heuristics. Also a moving target as developers add new code daily⇒ need automation

What they did: Heureka [CGO 2024] — automatically discovers hidden heuristics in LLVM by instrumenting compiler functions, manipulating their outputs, and using differential testing to verify correctness.

Key results: Up to 11.6% binary size reduction (single heuristic), 19.5% (stacking multiple via random search) vs. -Oz baseline. Tested on 8K functions (~2% of LLVM) across 38 benchmarks.



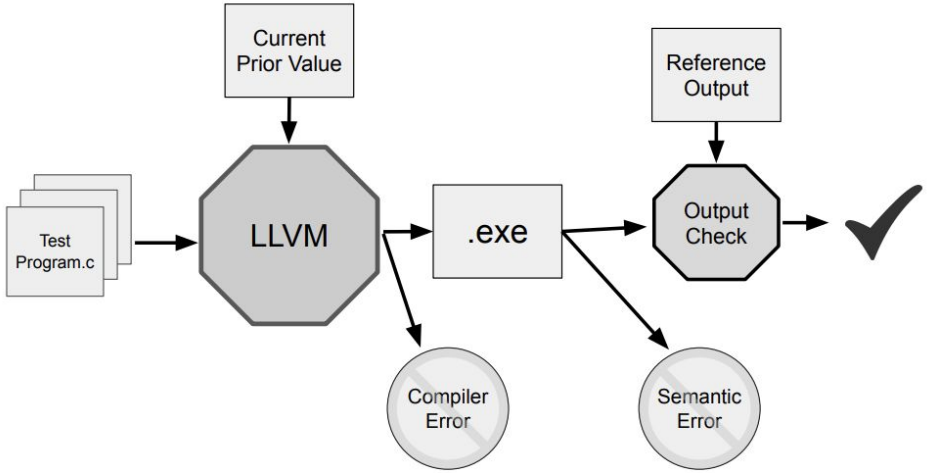
Revealing Compiler Heuristics through Automated Discovery and Optimization — CGO 2024

Heureka Pipeline:

1. Select a compiler function as candidate
2. Instrument it — wrap target functions with code to override their return values at runtime
3. Use differential testing to determine which value ranges the calling code accepts as valid function output

   Test: Run the program with the modified function and compare to an unmodified run. If outputs match → identified as tunable heuristic

4. Optimize a specific application using the discovered tuning targets: *randomly* combine multiple heuristics to get bigger binary size reductions than any single target could achieve alone



Searched 8K functions, found avg. 271 tuning targets per benchmark

- **Where it falls short:** After discovering heuristics, Heureka only uses naive random search (~6,400 samples) to combine them. This search was never open-sourced, and the authors called for more powerful methods.

Our Approach: Random Search + Bayesian Optimization

Heureka Outputs (targets + Priors) → **Random Search** → **Bayesian Optimization** → Compare

Step 1: Random Search

- Implement the Random Tuning Profile (RTP) described but not released in the paper

  RTP (repeated ~ 6400x per application): Randomly select a subset of tuning targets → randomly pick a fitted Prior for each target → sample a value from each Prior range → compile → measure binary size

- Serves as our baseline for comparison

Step 2: Bayesian Optimization (BO)

- Builds a surrogate model (Gaussian Process) that learns which heuristic values → smaller binaries
- Uses acquisition function (Expected Improvement) to pick the next configuration to try
  - Balances initial exploration (try unknown regions) with later exploitation (refine promising ones) → finds better results with fewer evaluations
  - Random search samples uniformly, which is wasteful
- BO is ideal here because each evaluation (compile + run + validate) is expensive, and the search space is continuous with manageable dimensions.

Evaluation

Task: tune LLVM heuristics with Heureka for individual applications

- Identify compiler tuning targets that generate savings by <u>themselves</u> and <u>in combination</u> using a random search approach
- Optimization Objective: Binary File Size Reduction
- Baseline : unmodified Vanilla Clang Compiler
- NAS and Polybench/C benchmark suites

Results

- **7.7% average** binary size reduction using single heuristics
- **Up to 11.6%** reduction on best-case benchmark (P-cor)
- **271 tuning targets** identified per benchmark on average

Results

- Combining multiple heuristics via random search → **up to 19.5%** reduction
- Restricted Set runs in **under 1 hour** on a single machine (vs. ~5 hrs full search)
- 4 heuristic types found: Obvious, Indirect, Data Field, and Unsafe

What Heureka Discovered

Four categories of compiler heuristics identified across 8,000 LLVM functions

Obvious Heuristic

getInlineCost

Clearly a cost function — compiler devs can adopt Heureka's value ranges directly

Indirect Heuristic

isNoBuiltin → 7.9% savings

Not designed as a heuristic, but toggling it stops printf→puts replacement, shrinking binaries

Data Field Heuristic

CFGSimplifyPass.BonusInstThreshold

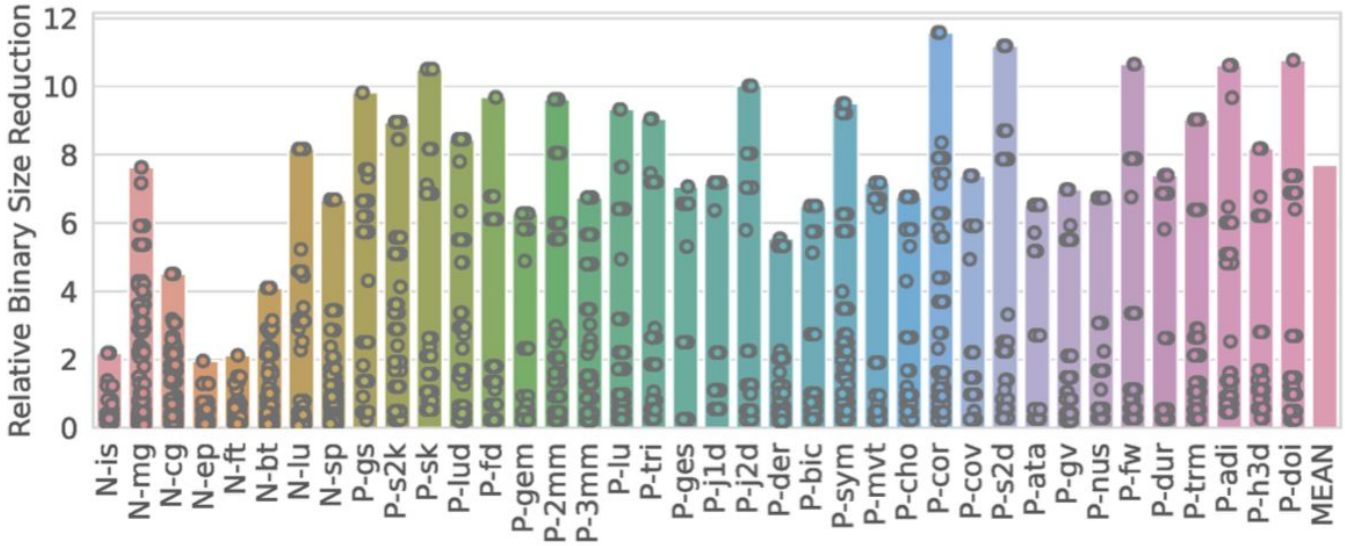
Heureka tracks object field side-effects, not just return values

Unsafe Heuristic

getLibFunc enum → 11.6% savings (memory leaks)

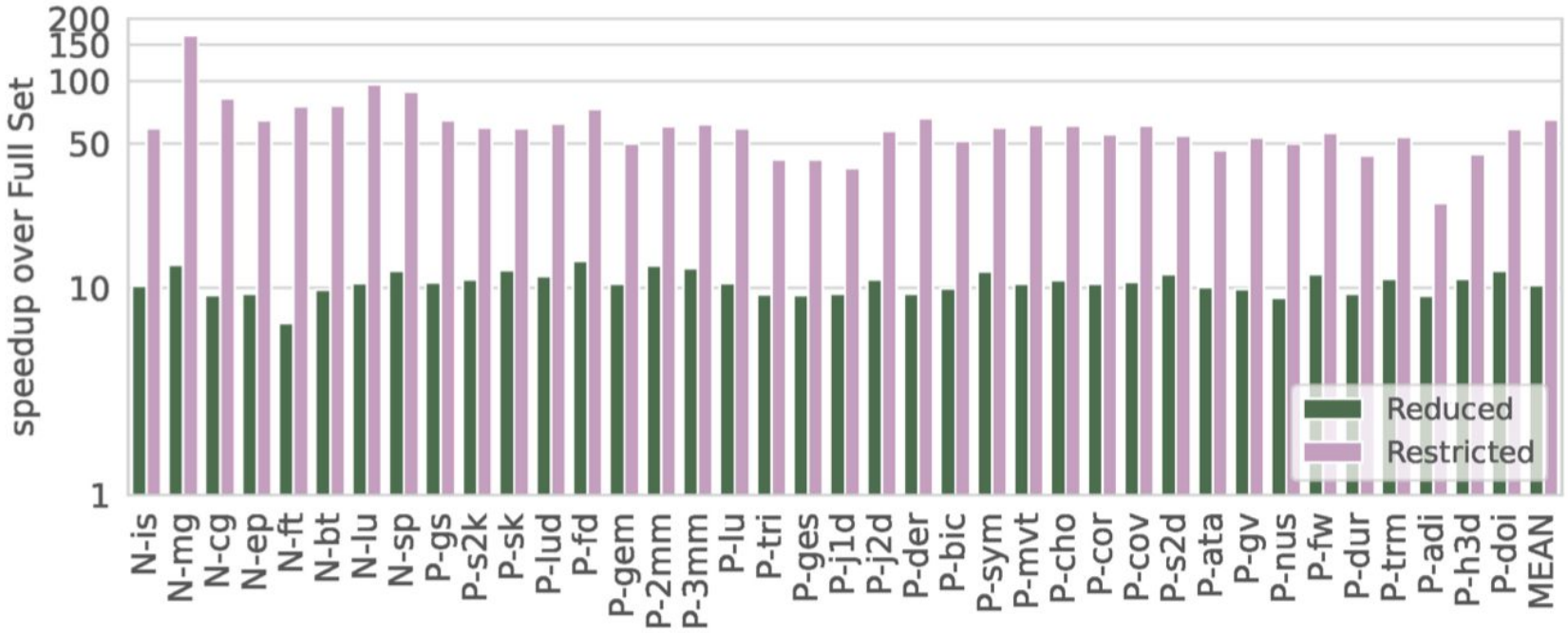
Produces savings but removes free() calls — needs expert review before use

Results



(b) Maximum binary size reduction found during heuristic search relative to `-Oz` baseline when considering single heuristics only. Overlaid circles indicate all of the tuning targets we could identify for this benchmark at their maximum observed binary size reduction.

Results



(b) Speedups of Reduced and Restricted Sets over runtime of the Full Sets on the cluster for each application on a log scale.

Results

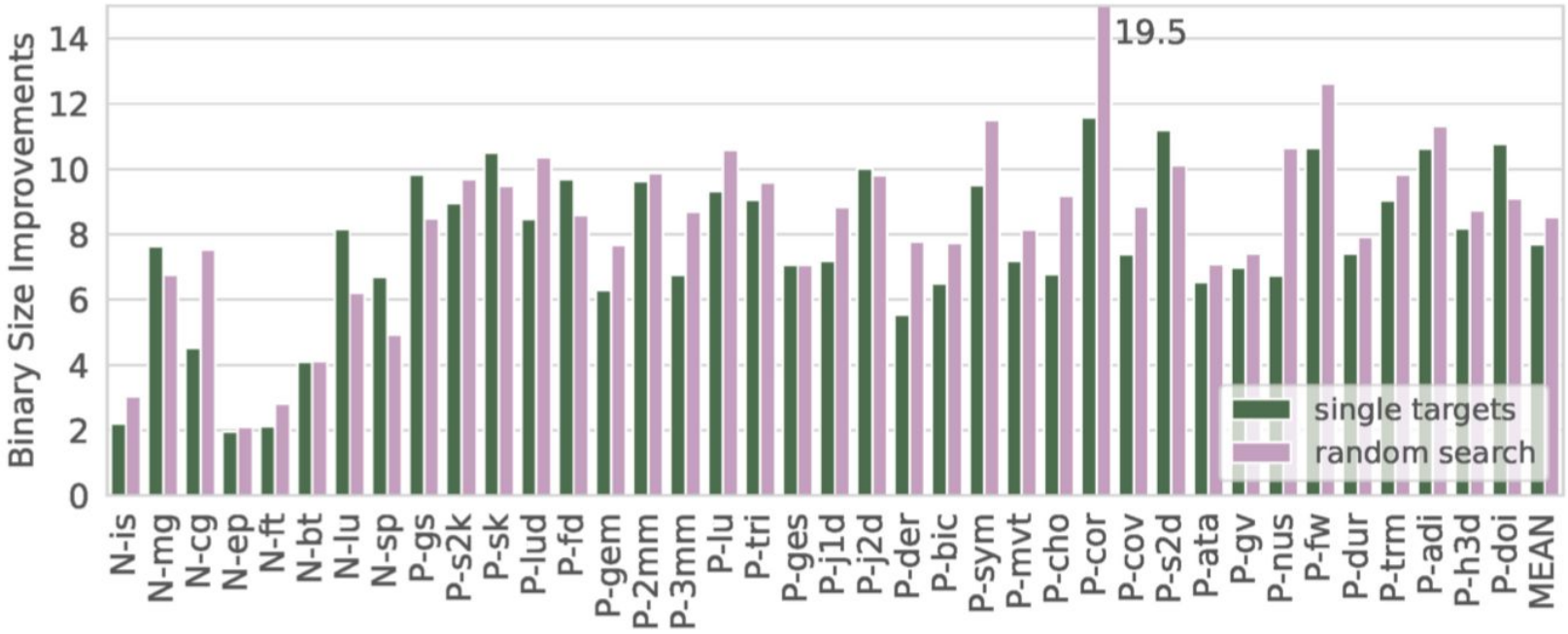


Fig. 9. Relative binary size reduction when combining multiple tuning targets for the Full Evaluation Set using a random search approach.

Weaknesses

Scope & Coverage Limitations	Correctness & Safety Limitations
Only small portion of LLVM explored	No correctness guarantees
Objective function is narrow	Weak random search to stack heuristics
Uses limited benchmark tests	

Conclusions

Novel Work: Automating heuristics discovery

Clean Core Insight:

If you can modify a function's output and the program still works correctly, you've found a heuristic

Strong Results:

7.7% binary size reduction on avg

19.5% peak result

Potential Research Impact

This work could influence

- Compiler autotuning
- Search-based software engineering
- ML-driven compilers
- Large-scale software observability

Potential Work: Automated discovery + ML optimization